

ScribeAI: A Course-Aware AI Lecture Companion with Retrieval-Augmented Generation

Daniel Chen*

Department of Computer Science

The City College of New York – CUNY
New York, USA

daniel.chen73@stu-mail.ccny.cuny.edu

Jia Xiang Zhang*

Department of Computer Science

The City College of New York – CUNY
New York, USA

jiaxiang.zhang77@stu-mail.ccny.cuny.edu

Zhigang Zhu

Department of Computer Science

City College and Graduate Center – CUNY
New York, USA

zzhu@ccny.cuny.edu

Abstract—College students face a fundamental cognitive bottleneck during lectures: they cannot simultaneously listen, comprehend, and take detailed notes. This problem is especially severe for students with disabilities who depend on peer notetakers, a system that is inherently unreliable. Existing commercial tools such as Genio and Jamworks offer transcription capabilities but lack course-specific AI, unified study workflows, and affordability. We present ScribeAI, a free, open-source, real-time lecture companion that integrates live transcription, a course-aware AI chatbot powered by Retrieval-Augmented Generation (RAG), automated structured summaries, and multi-format quiz generation with AI grading into a single session-based workflow. ScribeAI’s RAG pipeline extracts text from uploaded course materials, chunks them into segments, generates vector embeddings, and stores them in a knowledge base. At query time, cosine similarity search retrieves relevant chunks to ground the AI’s responses in actual course content. The system also features built-in accessibility controls including adjustable text size, color theme switching, and read-aloud speed settings, reflecting its co-design with the CCNY AccessAbility Center. Built with Next.js, Firebase, and Google Gemini, ScribeAI operates at zero cost on free-tier infrastructure and supports real-time multi-device synchronization. A user evaluation survey with 12 CCNY students yielded average feature ratings above 4.0/5.0 across all four core features, with Live Transcription scoring highest at 4.58/5.0. A video demonstration of ScribeAI is available at <https://youtu.be/aWuQqcZKtHA>.

Index Terms—Retrieval-Augmented Generation, assistive technology, lecture transcription, AI in education, accessibility, universal design for learning

I. INTRODUCTION

ScribeAI is an AI-powered, real-time lecture companion designed to help college students capture, understand, and retain lecture content more effectively. The idea grew out of a well-documented cognitive challenge in higher education. Research on note-taking and cognitive load theory has consistently shown that students face a fundamental dual-task problem: they must simultaneously comprehend spoken content and produce written notes, which competes for limited working

memory resources. Jansen et al. [1] identified five distinct cognitive processes involved in note-taking, each imposing load on working memory. Mueller and Oppenheimer [2] demonstrated that laptop note-takers tend toward verbatim transcription rather than deeper processing, which is detrimental to conceptual understanding. This cognitive bottleneck affects an estimated 83% of students [3] and is especially severe for students with disabilities who depend on peer notetakers, a system that is inherently unreliable.

At The City College of New York (CCNY), a senior public college within CUNY serving one of the most diverse urban student populations in the United States, the AccessAbility Center (AAC) provides accommodations through peer notetakers and licensed software. The two primary tools currently in use are Genio (formerly Glean) [10] and Jamworks [11]. However, these tools have significant limitations: Genio provides audio recording with transcript sync and basic AI-generated outlines but lacks an AI chatbot, course-specific grounding, or quiz generation. Jamworks offers recording with AI-generated chapter notes and flashcards but locks its chatbot behind a roughly \$19/month premium subscription and has no course-aware AI. Table I presents a detailed feature comparison. Neither tool was built for CCNY’s specific student population, many of whom are first-generation college students, English language learners, or students with learning challenges, and neither offers the kind of unified workflow where a student can transcribe, ask questions, get a summary, and quiz themselves all within a single lecture session.

The emergence of Retrieval-Augmented Generation (RAG) [4] has created new possibilities for educational AI. RAG grounds large language model outputs in retrieved external knowledge, significantly reducing hallucination and improving factual accuracy. Recent surveys of RAG applications in education [5], [6] confirm that RAG-based tutoring systems produce more accurate, context-specific responses than generic chatbot interactions, particularly when the retrieval corpus is drawn from the student’s actual course materials.

In this paper, we present ScribeAI, a free, open-source, real-time lecture companion that integrates live transcription, course-aware AI chat (RAG), automated summaries, multi-format quiz generation, and built-in accessibility controls into

This work is supported by the National Science Foundation through the AI Research Institutes program Award No. DMR-2433348, the CCNY CEN Course Innovation Grant, Disability and Accessibility Services at the CUNY Central Office of Student Affairs, and the CCNY Zahn Innovation Center.

*These authors contributed equally to this work.

one unified session. ScribeAI was co-designed with the CCNY AccessAbility Center and aligns with Universal Design for Learning (UDL) principles [7]. A user evaluation with 12 CCNY students yielded ratings above 4.0/5.0 across all core features. Our contributions include:

- 1) A full RAG pipeline that indexes syllabi, slides, textbooks, and images into a vector knowledge base, enabling course-grounded AI responses rather than generic outputs.
- 2) A unified session workflow that eliminates context-switching between separate transcription, note-taking, chat, and study tools.
- 3) Built-in accessibility features including adjustable text size, color themes (light, dark, high-contrast), read-aloud speed control, and reduced-motion settings, co-designed with the CCNY AccessAbility Center.
- 4) A zero-cost architecture deployed on Firebase Spark and Vercel Hobby tiers, ensuring accessibility for all students regardless of budget.

This paper is organized as follows. After the introduction in Section I, Section II presents related work on note-taking, assistive technology, and RAG. Section III summarizes the rationale of our solution. Section IV describes the system architecture and implementation. Section V details the user evaluation. Section VI concludes the paper and outlines future directions.

II. RELATED WORK

A. Note-Taking and Cognitive Load

Research on note-taking has established that the act involves five competing cognitive processes: comprehending, identifying key points, linking to prior knowledge, paraphrasing, and transcribing [1]. Bui and Myerson [9] showed that working memory capacity significantly affects note-taking quality, with students who have lower working memory capacity producing substantially less complete notes. Shi et al. [3] found that roughly 83% of college students report difficulty absorbing lecture content while simultaneously writing notes. Mueller and Oppenheimer [2] further demonstrated that laptop note-takers tend toward verbatim transcription rather than meaningful synthesis. These findings collectively establish a clear need for tools that offload the transcription burden so students can focus on comprehension.

B. Assistive Technology and Universal Design

Universal Design for Learning (UDL) [7] provides a framework for creating flexible learning environments that accommodate diverse learners by providing multiple means of engagement, representation, and action/expression. Zhang et al. [8] analyzed UDL in U.S. federal education law, finding growing policy support for universally designed technologies that proactively serve students with and without disabilities. ScribeAI was designed with UDL principles at its core: providing multiple means of accessing lecture content through live transcript, AI chat, structured summaries, and interactive

quizzes, so that every student can engage with material in the way that works best for them.

Beyond curricular flexibility, digital accessibility depends on concrete interface accommodations. The Web Content Accessibility Guidelines (WCAG) recommend adjustable text sizing, sufficient color contrast, screen-reader compatibility, and respect for reduced-motion preferences. In practice, however, most lecture-capture tools delegate these needs to browser or operating-system defaults rather than exposing dedicated in-app controls. This gap disproportionately affects students with low vision, dyslexia, or vestibular sensitivities, who benefit from being able to tune the interface itself. ScribeAI therefore treats these accommodations as first-class features, implemented as the built-in accessibility controls detailed in Section IV.

C. Retrieval-Augmented Generation

Lewis et al. [4] introduced the RAG framework for knowledge-intensive NLP tasks, demonstrating that grounding language model outputs in retrieved documents significantly improves factual accuracy. Li et al. [5] surveyed RAG applications specifically in education, finding that course-specific retrieval significantly improves response quality over generic chatbot interactions. Zhao et al. [6] provided a comprehensive taxonomy of RAG architectures. Our work applies these principles to build a lecture companion that grounds all AI interactions in the student’s actual course materials.

D. Existing Lecture Capture Tools

Several commercial tools address the lecture capture problem. Genio [10] (formerly Glean) provides audio recording with transcript sync and basic AI-generated outlines, but lacks an AI chatbot, course-specific grounding, quiz generation from transcripts, or multi-device sync. Jamworks [11] offers recording with AI-generated chapter notes and flashcards, but its chatbot is locked behind a premium subscription and it has no course-aware AI or real-time sync. Table I presents a detailed feature comparison of ScribeAI with both tools. Neither was designed for CCNY’s student population, and neither provides the unified workflow that ScribeAI offers, which will be further detailed in the next section.

TABLE I
FEATURE COMPARISON WITH EXISTING TOOLS

Feature	Genio [10]	Jamworks [11]	ScribeAI
Live Transcription	Yes	Yes	Yes (Gemini)
AI Chatbot	No	Premium	Yes (RAG)
Course-Aware RAG	No	No	Yes
Quiz Generation	Basic	Flashcard	MCQ+Short+FIB
AI Grading	No	No	Yes (Gemini)
Multi-Device Sync	No	No	Real-time
Accessibility Controls	Limited	Limited	Built-in
Teacher Dashboard	No	No	Yes
Cost to Student	Inst. \$\$	\$19/mo	Free

III. RATIONALE OF THE SOLUTION

We made several deliberate architectural and design decisions early on that shaped how ScribeAI differs from existing

tools. To contextualize these decisions, Table I presents a direct feature comparison between ScribeAI and the two primary tools currently available at CCNY. These features are summarized below.

a) *Unified session workflow*: The biggest frustration we heard from users of Genio and Jamworks was the fragmented experience: record in one place, review notes in another, quiz yourself with a third tool. ScribeAI collapses all of this into a single lecture session. You open a session, hit record, and immediately have access to live transcription, an AI chatbot grounded in your course materials, auto-generated summaries, and quiz generation. There is no context switch. Everything lives in one place and refers to the same lecture.

b) *Course-aware RAG pipeline*: Most AI lecture tools use generic language models that know nothing about your specific course. ScribeAI implements a full RAG pipeline: when students upload their syllabus, lecture slides, textbooks, or other materials, our system extracts text, chunks it into manageable segments, generates vector embeddings for each chunk, and stores them in Firestore. When the AI chatbot receives a question, it performs a cosine similarity search against the knowledge base to retrieve the most relevant chunks and includes them as context in the prompt.

c) *Real-time transcription with chunked audio processing*: Rather than relying on post-session transcription, ScribeAI transcribes audio in real time using Google’s Gemini 3.1 Flash Lite model. Audio is captured in 5-second chunks from the browser’s MediaRecorder API and sent to Gemini for transcription. This chunked approach avoids the instability of long-running streaming connections and provides reliable, near-instant transcription throughout the lecture.

d) *Free and open-source*: We intentionally built ScribeAI on a zero-cost stack. Firebase’s Spark plan, Gemini’s free tier through Firebase AI, and Vercel’s hobby hosting tier mean the application costs nothing to run at its current scale. This aligns with UDL principles [7] that accessibility tools should not create additional barriers through cost.

e) *Multi-device sync*: One of the most requested features from our user survey was mobile access. Students wanted to record on their phone while reviewing the transcript on their laptop. We implemented real-time sync using Firestore’s onSnapshot listeners, so transcript updates, summaries, and quiz state push instantly across all logged-in devices.

f) *Built-in accessibility*: Accessibility was a first-class design goal rather than an afterthought. Co-designed with the CCNY AccessAbility Center, ScribeAI ships with an in-app accessibility panel offering adjustable text size, light/dark/high-contrast color themes, read-aloud speed control, and a reduce-motion toggle. These controls persist across sessions and apply globally, so students who rely on larger text, higher contrast, or a particular screen-reader pace can tailor the whole application to their needs without depending on browser defaults. This directly addresses the interface-level accommodation gap left by Genio and Jamworks.

IV. SYSTEM ARCHITECTURE

ScribeAI is a full-stack web application (Fig. 1) built with Next.js 16 using the App Router architecture with TypeScript, styled with Tailwind CSS and Radix UI components. The backend runs entirely on Firebase: Firestore for real-time document storage with over 40 exported API functions, Firebase Auth for user management with role-based access, and Firebase Storage for file uploads. The AI engine uses Google Gemini 3.1 Flash Lite Preview accessed through the Firebase AI SDK, with a separate embedding endpoint using the Google Generative AI SDK for the RAG pipeline. The application is deployed on Vercel with automatic deployments from GitHub, achieving zero operational cost on Firebase’s Spark plan, Gemini’s free tier, and Vercel’s Hobby plan. In the following, we will describe the complete feature workflow within a single lecture session, illustrating how transcription, AI chat, summaries, and quiz generation all operate from a unified context.

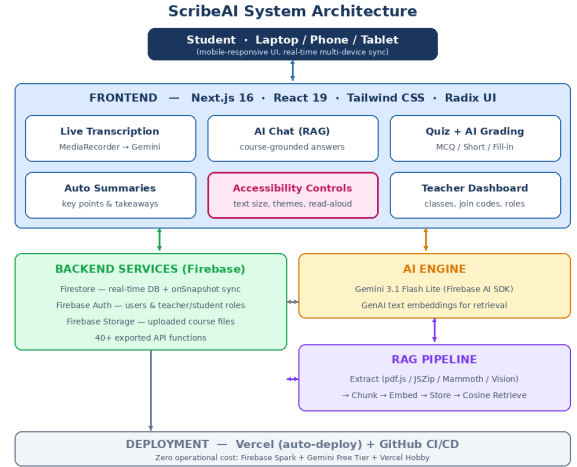


Fig. 1. ScribeAI system architecture. The frontend hosts the six feature modules described in this section—live transcription, AI chat (RAG), quiz generation with AI grading, auto summaries, accessibility controls, and the teacher dashboard—served by Firebase backend services, the Gemini AI engine, and the RAG pipeline. All components operate at zero cost on Firebase Spark, Gemini Free Tier, and Vercel Hobby.

A. RAG Pipeline

The RAG pipeline processes uploaded course materials through four stages. First, text extraction handles multiple file formats: pdf.js for PDFs, JSZip for PPTX slide XML, Mammoth for DOCX, and Gemini’s vision model for images. Second, the extracted text is chunked into approximately 1000-character segments to balance retrieval granularity with context completeness. Third, each chunk is embedded using Gemini’s text embedding model to produce dense vector representations. Fourth, the chunks and their embeddings are stored in Firestore, forming the class-specific knowledge base.

At query time, when a student sends a message to the AI chatbot, the system generates an embedding for the question, performs cosine similarity search against all knowledge chunks

for that class (including uploaded materials and the live transcript), retrieves the top relevant matches, and includes them as context in the Gemini prompt along with the conversation history. This ensures the AI’s responses are grounded in the student’s actual course content rather than relying solely on the model’s parametric knowledge.

B. Real-Time Transcription

The transcription system captures audio through the browser’s MediaRecorder API and segments it into 5-second chunks. Each chunk is converted to base64 and sent to Gemini 3.1 Flash Lite for transcription. This chunked approach was a deliberate engineering decision: long-running audio streams are unreliable in browser environments due to connection drops, tab hibernation, and varying network conditions. The 5-second window provides a good balance between transcription latency and connection reliability, as each chunk is an independent request that can retry on failure. All connected devices receive updates instantly via Firestore’s `onSnapshot` listeners, enabling a key use case: students can record on their phone while reviewing the growing transcript on their laptop.

C. Quiz Generation and AI Grading

ScribeAI generates quizzes directly from the lecture transcript in three question types: multiple choice (MCQ), short response with AI grading, and fill-in-the-blank. For short response questions, the system implements a full AI grading pipeline: Gemini evaluates the student’s answer against a sample correct response, assigns a score, and provides a detailed explanation. This gives students immediate, substantive feedback without requiring instructor involvement.

D. Teacher Dashboard and Class Management

To support institutional adoption, we built a role-based system with teacher and student accounts. As shown in Fig. 2, teachers can create classes with auto-generated join codes, view enrolled students, share live lecture sessions, and manage course materials. Students join classes via the code and gain access to shared sessions and the class knowledge base. This feature was specifically designed with the CCNY AAC use case in mind: an instructor or AAC coordinator creates a class, generates a join code, and distributes it to students with accommodations, replacing the unreliable peer notetaker system.

E. Accessibility

ScribeAI integrates built-in accessibility controls co-designed with the CCNY AccessAbility Center, shown in Fig. 3. These include adjustable text size via a continuous slider (scaling all interface text proportionally), three color themes (light, dark, and high-contrast), read-aloud speed control for screen reader compatibility, and a reduce-motion toggle that minimizes animations for users with vestibular sensitivities. These settings persist across sessions and apply globally across all pages. Unlike Genio [10] and Jamworks [11], which rely on browser-level accessibility or offer no dedicated

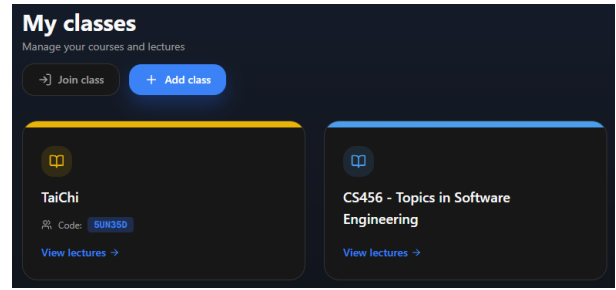


Fig. 2. Class management interface showing teacher-created classes with color-coded cards and join codes. Students enter a code to access shared lectures, AI chat, and the course knowledge base.

controls, ScribeAI treats accessibility as a first-class feature embedded directly in the application.

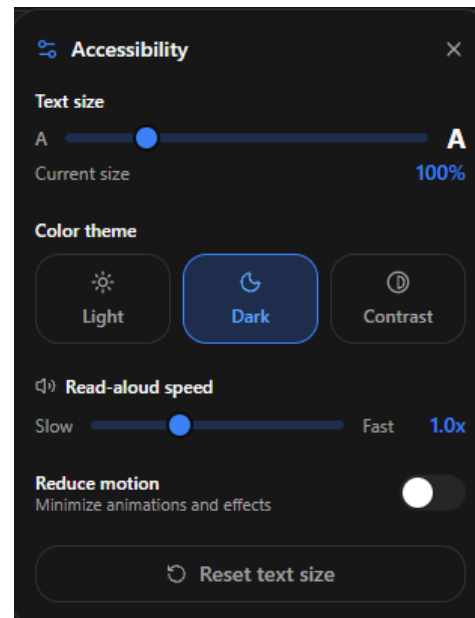


Fig. 3. Built-in accessibility settings panel with text size slider, color theme selection (light, dark, contrast), read-aloud speed control, and reduce-motion toggle.

F. Mobile-Responsive Design

After our user evaluation revealed strong demand for mobile access, we implemented a fully responsive experience using Tailwind CSS responsive breakpoints. Fig. 4 compares the desktop and mobile layouts. On desktop, the lecture session displays the transcript in a central panel with AI chat pinned as a sticky sidebar. On mobile, the AI chat slides in as an overlay, keeping the full-width transcript view uncluttered.

This responsive design, combined with Firestore real-time sync, enables the dual-device workflow that students requested: record on phone, review on laptop, with transcript, summary, and quiz state all synchronized instantly.

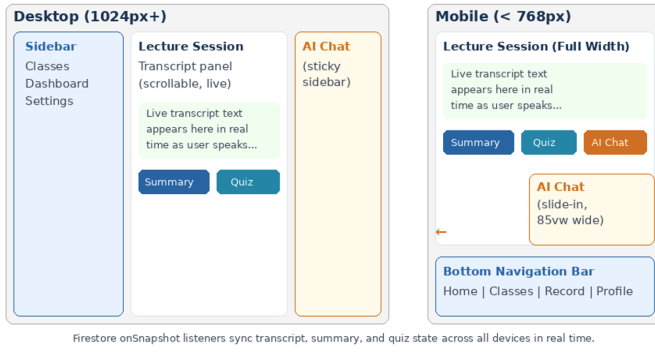


Fig. 4. Desktop vs. mobile responsive layout. On desktop, AI chat pins as a sidebar; on mobile, it slides in as an overlay. Firestore `onSnapshot` listeners sync all data across devices in real time.

V. EVALUATION

A. User Survey Methodology

We conducted a user evaluation survey using Google Forms, distributed to classmates, peers, and contacts at CCNY. We received 12 responses from students across multiple departments. The survey consisted of quantitative ratings of ScribeAI's four core features on a 1 to 5 Likert scale (1 = not useful, 5 = extremely useful), and open-ended questions about strengths, areas for improvement, and willingness to use the tool.

B. Quantitative Results

Results (Fig. 5) were strong across the board. Live Transcription received the highest average rating at 4.58/5.0, confirming that real-time transcription is the most valued capability. Quiz Generator scored 4.33, AI Chatbot scored 4.17, and Syllabus Upload scored 4.08. All four features exceeded 4.0/5.0, validating the unified, course-grounded design approach.

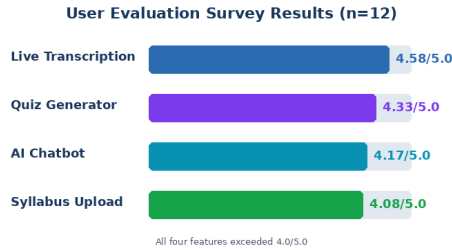


Fig. 5. User survey results from 12 CCNY student respondents. All four core features scored above 4.0/5.0, with Live Transcription rated highest.

C. Qualitative Feedback

Open-ended responses provided valuable insights. Users described the application as “intuitive,” with one respondent noting the transcription “feels so smooth recording in real time.” Constructive feedback centered on three themes: mobile access (motivating our responsive redesign in Section IV-F), more professional visual design (addressed through color-coded UI elements), and more quiz variety (leading us to implement short response and fill-in-the-blank types).

VI. CONCLUSIONS AND FUTURE WORK

We presented ScribeAI, a free, open-source lecture companion that unifies real-time transcription, course-aware AI chat via RAG, automated structured summaries, multi-format quiz generation with AI grading, and built-in accessibility controls into a single session-based workflow. By implementing a full RAG pipeline that indexes course materials into a searchable vector knowledge base, ScribeAI ensures every AI interaction is grounded in the student's actual course content. The system was co-designed with the CCNY AccessAbility Center, aligns with UDL principles, and operates at zero cost. User evaluation with 12 CCNY students confirmed strong reception, with all features rated above 4.0/5.0.

Future directions include: (1) multi-language transcription support to serve CCNY's linguistically diverse student body; (2) Anki flashcard export for spaced repetition study; (3) deeper teacher analytics including engagement heatmaps and topic coverage gap analysis; (4) expansion to other CUNY campuses through institutional licensing; and (5) a native mobile application with offline recording capabilities.

ACKNOWLEDGMENTS

We thank the NSF AI Materials Institute (AI-MI) for supporting the CCNY BEAT+ Capstone Program, and the CCNY AccessAbility Center for their partnership throughout the design and evaluation process. We also thank Chris Bobko at the Zahn Innovation Center for entrepreneurial mentorship, and Raymond Perez, Tayler Resnick, Shams Mamun and Carrie L. Shockley at the CUNY Central Office of Student Affairs for connecting us with cross-campus accessibility pathways.

REFERENCES

- [1] R. S. Jansen, D. Lakens, and J. de Nooijer, “An integrative review of the cognitive costs and benefits of note-taking,” *Educ. Research Review*, vol. 22, pp. 223–233, 2017.
- [2] P. A. Mueller and D. M. Oppenheimer, “The pen is mightier than the keyboard: Advantages of longhand over laptop note-taking,” *Psychological Science*, vol. 25, no. 6, pp. 1159–1168, 2014.
- [3] Y. Shi, H. Yang, Z. Yang, et al., “Examining the effects of note-taking styles on college students' learning achievement and cognitive load,” *Australasian J. Educational Technology*, vol. 38, no. 1, pp. 1–11, 2022.
- [4] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [5] X. Li, Y. Zhang, H. Wang, et al., “Retrieval-augmented generation for educational application: A systematic survey,” *Computers and Education: Artificial Intelligence*, vol. 9, 2025.
- [6] P. Zhao, H. Zhang, Q. Yu, et al., “Retrieval-augmented generation for AI-generated content: A survey,” *arXiv preprint arXiv:2402.19473*, 2024.
- [7] CAST, “Universal Design for Learning Guidelines version 2.2,” 2018. [Online]. Available: <http://udlguidelines.cast.org>
- [8] L. Zhang, R. A. Carter, and N. J. Hoekstra, “A critical analysis of universal design for learning in the U.S. federal education law,” *Policy Futures in Education*, vol. 22, no. 2, pp. 178–197, 2024.
- [9] D. C. Bui and J. Myerson, “The role of working memory abilities in lecture note-taking,” *Learning and Individual Differences*, vol. 33, pp. 12–22, 2014.
- [10] Glean (formerly Sonocent), “Glean – Note Taking Tool for Students,” 2026. [Online]. Available: <https://glean.co>. [Accessed: Jul. 8, 2026].
- [11] Jamworks, “Jamworks – AI Note Taker for Students,” 2026. [Online]. Available: <https://jamworks.com>. [Accessed: Jul. 8, 2026].